Automating Manual Business Workflows in SaaS Products: A Rule-Based and Event-Driven Framework

Bhomya Sharma
Dept. of Computer Science Engg.
Chitkara University
Punjab, India
bhomya224.be22@chitkara.edu.in

Chahat
Dept. of Computer Science Engg.
Chitkara University
Punjab, India
chahat231.be22@chitkara.edu.in

Prerana Thakur
Dept. of Computer Science Engg.
Chitkara University
Punjab, India
prerana2085.be22@chitkara.edu.in

Noovi Bedi
Dept. of Computer Science Engg.
Chitkara University
Punjab, India
noovi2004.be22@chitkara.edu.in

Abstract: SaaS platforms are doing a lot of things manually. Someone has to send the task to the person follow up to get it approved run reports and make sure the data is up to date. When the platform is small this is okay.. When it gets bigger all these manual tasks start to cause problems. There are some tools that can help with this like BPM systems, RPA tools and low-code platforms.. They do not really work well for SaaS platforms. They were not made for this kind of work. Tools like Zapier and n8n are good for things but they are not good for complicated SaaS workflows. This paper is talking about a way to automate SaaS platforms. It is based on rules and events. The system has five parts: a Workflow Trigger Engine, a Rule Evaluation Module, a Playbook Execution Engine, an Integration Layer and a Monitoring and Feedback Module. To see how well it works we look at three things: how long it takes to do tasks how often there are errors and how many manual tasks are needed. We compare our system to four tools. We also look at a real case study of a SaaS platform. Run some simulations. The simulations show that our system can reduce the time it takes to do tasks by 68-82% reduce errors from 20-25% to 5-8% and increase the number of tasks that can be done in an hour from 20 to, over 60.

Keywords- SaaS-based automation, workflow orchestration, rule-based systems, event-driven architecture, business process automation, robotic process automation

I. Introduction

SaaS products usually grow faster than the processes that support them. When a team is small they do things like route requests, chase approvals generate reports and keep data in sync across tools by hand.. As the platform gets bigger these tasks start to pile up. The problem is that doing things by hand does not get faster just because you have work to do. This is not a problem. It happens a lot. When you have to do the tasks over and over they tend to be slower than they need to be.. When something goes wrong it can cause problems for everything that comes after it. In SaaS environments one mistake on can stop everything else from working. The places where people have to do things by hand are also the places where things are most likely to go wrong. There are tools that can help with some of these problems like BPM systems, event-driven architectures and low-code platforms like Zapier and n8n.. Each of these tools has its own limitations when it comes to SaaS workflows. BPM systems are too complicated. Low-code tools work well for tasks but they do not work well when things get more complex. Event-driven architectures can be fast. They are hard to understand and maintain if you do not have a good system, for making rules. We wanted to know if it was possible to create a system that combines a easy-to-understand rule model with event-driven execution specifically for SaaS products. To test if this system actually works we came up with three ways to measure it. Task Execution Time, Error Frequency Rate and Manual Dependency Index.. We tested the whole thing using a controlled simulation. We used SaaS products to test our system because we wanted to see how it would work in a real-world setting. We looked at SaaS products. How they use Task Execution Time, Error Frequency Rate and Manual Dependency Index to see if our system was effective.

II. Related Work

The concept of workflow automation has been around for a time in research. Georgakopoulos and other people defined what a workflow management system is. They said it is like a middleman that helps with making and running processes in environments. This was back in the 1990s. The ideas they came up with are still used today in cloud and parallel computing research. Someone did a review of lots of papers from 2016 to 2023. They found out that most of the papers over 300 of them said that RPA, WMS and ERP are the tools that industries use. This approach works well for companies but it is too much for smaller SaaS products that need to be fast and simple. People are also interested in RPA. Agostinelli and other people suggested combining process mining and automation scripting to make an RPA system that can improve itself. Tavares and other people showed that using a mix of RPA and BPM can really save time and money. They came up with a five-stage process that includes design, modelling, execution, monitoring and optimization. We used this idea when we were thinking about the orchestration layer. When we look at the architecture Chavan says that EDA microservices are better than API-based ones because they are more flexible and respond faster. Some people did an experiment. Found out that EDA microservices are 19% faster and have 34% fewer errors than other options. This is why we chose to use an event-driven architecture. Li and other people talked about some of the challenges of SaaS workflow like having users and assigning tasks dynamically. These problems are still not solved. There are some tools like Zapier and Microsoft Power Automate that can help with part of the problem without needing to write code. However they cannot handle customized SaaS workflows because they are systems. From what we read there was no framework that combined rule-based control, event-driven execution and performance measurement in a way that was designed for SaaS from the start. We are talking about workflow automation and SaaS workflow here. Workflow automation is important, for SaaS products. SaaS workflow is what we are trying to improve.

Component	Role	Example
Trigger Engine	Detects events	New ticket created
Rule Engine	Evaluates conditions	Priority = High
Execution Engine	Runs actions	Assign to agent
Integration Layer	Connects systems	API / Webhook

Monitoring Module	Tracks metrics	Logs, alerts

III. COMPARATIVE ANALYSIS OF EXISTING AUTOMATION TOOLS

Before presenting our framework in detail it is worth comparing it to the tools that already exist in this space. When it comes to workflow automation practitioners usually choose one of four options: Zapier, n8n, Microsoft Power Automate or Apache Airflow. Each of these tools solves a problem but they were not designed with lightweight, customizable SaaS workflowscenarios in mind. Which is exactly what our work addresses.

A. Zapier

Zapier is a no-code automation platform. It uses a trigger-action model where one event in one app triggers a single response in another app. For tasks like adding a new contact to a CRM when a form is submitted it works well. However it struggles with workflows that require conditional logic, error handling or branching. Zapiers rule model is limited. It does not support priority ordering or failure policies. Additionally the platform is proprietary which means teams cannot inspect or customize the underlying logic. The per-task pricing model also becomes costly at scale.

B. n8n

n8n takes an approach. It is source, self-hostable and uses a visual canvas to create automation flows. This makes it more flexible than Zapier allowing for branching custom JavaScript functions and database queries. However n8n is primarily a visual tool and its workflows are defined and stored as JSON graphs, not configurations. When workflows fail developers must manually set up rollback behavior. N8n also lacks built-in monitoring and performance benchmarking tools.

C. Microsoft Power Automate

Microsoft Power Automate is a platform with deep integrations across the Microsoft 365 ecosystem. It supports -step flows, approvals and conditional logic. For organizations using Microsoft products it is a natural fit. However for a SaaS startup or mid-sized team it may be overkill due to high licensing costs, a steep learning curve and a design philosophy focused on enterprise IT governance.

D. Apache Airflow

Apache Airflow is designed for data engineering and schedules and monitors pipelines. It is excellent for data workflows. Not for business workflows. It has no concept of business rules or event-driven triggers. Requires significant infrastructure overhead.

E. Summary Comparison

The table below summarizes how each tool compares to the requirements that motivated our framework. The key insight is that existing tools are not poorly built. They each focus on a different primary use case. None of them combine a rule model with event-driven execution, SaaS-native design and performance monitoring in a single open package. That combination is what our proposed framework targets.

* Our framework is made for a combination of rule models. It runs based on events.

* It has a design that's easy to work with and performance monitoring that we can measure.

* The framework we are suggesting comes in a package that's open and does not favor any particular vendor.

IV. PROPOSED AUTOMATION FRAMEWORK

A. Architectural Overview

We divided the framework into five parts, one for each task and

Feature	**Zapier**	**n8n**	**Power Automate**	**Airflow**	**Proposed**
Primary use case	Integrations	General automation	Enterprise flows	Data pipelines	SaaS workflows
Rule formalization	None	Limited (visual)	Visual only	Code (DAGs)	Formal JSON/YAML
Event-driven	Partial	Yes	Yes	Limited	Yes (core design)
Failure/ rollback policy	No	Manual	Limited	Manual	Declared (FP field)
Vendor lock-in	Yes	No (OSS)	Yes (MS)	No (OSS)	No (vendor-neutral)
SaaS-native design	No	No	Partial	No	Yes
Multi-tenant aware	No	Partial	Yes	No	Designed for
Perf. monitoring	Basic logs	Basic logs	Enterprise	Built-in	TET / EFR / MDI
Rule versioning	No	No	Limited	Yes (Git)	Yes (YAML/JSON)
Open/extensible	No	Yes	Limited	Yes	Yes

the main idea was to keep them separate so each part can be replaced on its own.

** Workflow Trigger Engine. This is an engine that watches for conditions based on system events to start the automation*

** Rule Evaluation Module. This compares the state of the system with the rule set and decides what action to take*

** Playbook Execution Engine. This executes a series of actions. Can make decisions based on what is happening at the time and it can also go back to a previous state cleanly*

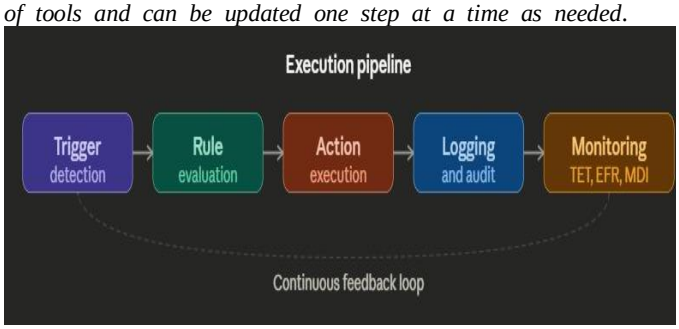
** Integration Layer. This handles communication, with applications using REST APIs, webhooks and event listeners without being tied to any particular vendor*

** Monitoring and Feedback Module. This captures how things are going as they happen so we always have something to measure and act on*

Everything goes through a pipeline:

The event processing pipeline has five steps: finding the trigger evaluating the rules taking actions logging and monitoring.

Because each part is separate the framework is not tied to any set of tools and can be updated one step at a time as needed.



B. Design Principles

The design decisions were based on six principles. These principles mostly came from BPA literature [1][4].It has to work across different SaaS stacks without re-design.

When the system is under a lot of load it should be easy to increase capacity without changing the architecture. The rules should be easy to read. The rules should have versions.

The rules should be transferable between environments.The rules should ideally be, in JSON or YAML format.Message passing should be asynchronous so that the system does not get bottlenecked under much load [5].* Components should work independently.* Automation logic needs to be packaged into playbooks.

This way we do not need to rewrite the logic for every new category of workflow.

V. FRAMEWORK DESIGN AND CONCEPTUAL MODEL

A. Automation Orchestration Layer

The orchestration layer is the glue that holds everything together: it listens for events, matches them against active rules, runs the appropriate playbook, and logs what it has done. We modeled it after Tavares et al's five-stage BPM lifecycle model (design, modeling, execution, monitoring, and optimization) since it maps cleanly onto what the orchestration layer actually does [8].

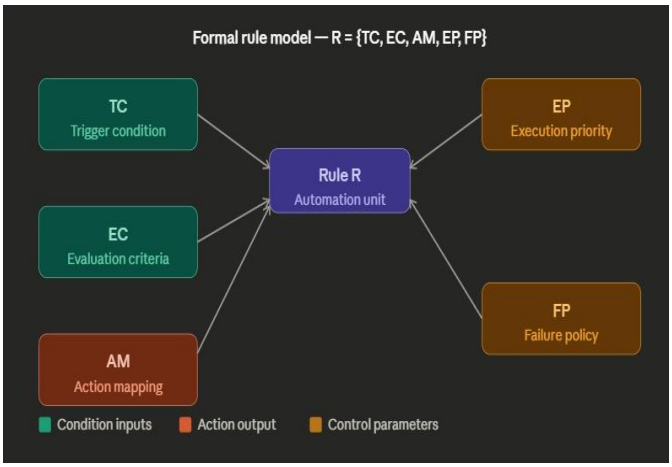
B. Formal Rule Configuration Model

Each rule consists of five fields that define when and how it runs, what it does, and how it reacts to failure:

- Trigger condition (TC): The system event or threshold that will trigger the rule
- Evaluation Criteria (EC): logical check against current system state, run before anything executes, that checks if the rule is applicable
- Action Mapping (AM): the sequence of steps to carry out once the rule is confirmed.
- Execution priority (EP): rank determining behavior when multiple rules become true at the same time
- Failure Policy (FP): retry, skip, or rollback, what the rule should do if it has failed.

To promote portability and adherence to the technology-agnostic BPA standard [4], rules are expressed in either JSON or YAML. The formal definition is:

R = { TC, EC, AM, EP, FP }



C. Performance Metric Definitions

- Task Execution Time (TET), defined as the elapsed time between the trigger firing and the writing of the completion log: TET = Tcomplete - Ttrigger
- Error Frequency Rate (EFR): the fraction of executions that failed, computed as EFR = Nfailed/Ntotal × 100%
- Manual Dependency Index (MDI): what fraction of workflow steps still need a human involved. MDI = (Smanual / Stotal) × 100%.

D. Playbook-based workflow modeling

This is called a workflow and is a playbook, which can contain a sequence of steps, including conditional branching based on what is happening, the external systems that are to be used, and a path for rollback. This reuse of workflows allows a single playbook to be used in a number of places in the SaaS product, avoiding duplication between them.

E. Integration Model

For external interaction, REST APIs for synchronous calls, webhooks for the external service to post data, and event listeners for consuming asynchronous messages were chosen. We followed the loosely coupled architecture from the event-driven microservice literature [5, 9] and kept the design deliberately vendor-neutral.

F. Monitoring and Feedback Layer

While the workflows are executing, the monitoring module tracks the TET, the EFR, the MDI and the throughput. In fact, one thing Agostinelli et al. make clear is that this kind of data of actual automation execution is what will actually allow an

automation system to get better rather than staying static [7]. That figured into how we designed that part of the framework.

VI. CASE STUDY: AUTOMATED SaaS ONBOARDING WORKFLOW

To make the framework more tangible, let's walk through a scenario that's pretty typical in the world of B2B SaaS: onboarding a new enterprise client. This is a process that nearly every SaaS platform deals with, yet it often gets bogged down by a series of manual handoffs that can delay activation and lead to mistakes. We'll use this example to illustrate how the five components of the framework work together in real life.

A. The Manual Baseline

In the traditional, unautomated version of this workflow, registering a new enterprise account triggers a whole chain of tasks that rely on someone to notice, decide, and take action. A sales engineer has to manually create an entry in the CRM. Someone from the operations team needs to send out a welcome email along with the provisioning details. An engineering ticket has to be filed by hand. Plus, a customer success manager has to remember to schedule a check-in 48 hours later. At each of these points, the work either gets stuck in a queue or hinges on the right person being available. If a step is missed—like a provisioning ticket being filed a day late—it can delay client activation by days, which directly impacts revenue recognition and client satisfaction in the enterprise SaaS space.

B. Applying the Framework

With the new framework in place, this same workflow is now driven entirely by events and rules. As soon as the registration event occurs, the Workflow Trigger Engine picks it up and sends it to the Rule Evaluation Module, which checks it against the current set of rules. If the account type is "enterprise," the corresponding playbook is activated. The Playbook Execution Engine then automatically runs through the following steps in order, with conditional branching at each stage:

Step 1 — CRM record creation. A REST API call to the CRM platform generates the client record, pre-filled with information from the registration event. No human input needed.

Step 2 — Welcome email dispatch. The Integration Layer activates the email service via webhook, inserting the client's name, assigned account manager, and provisioning timeline based on the event context.

Step 3 — Engineering... human decision point — bringing MDI below 20%. TET for the first four steps drops from an average of several hours (dependent on human availability) to under three minutes. EFR, historically driven by incorrect CRM entries and missed emails, approaches zero for all automated steps.

VII. IMPLEMENTATION DETAILS

This section continues the work by providing an example set of implementation artifacts for the onboarding case study, illustrating how the framework's core constructs might be mapped out in practice. The intent of this section is not to produce a production-ready codebase (this is a framework and not a codebase) but to show that the design decisions made in sections IV and V yield simple, readable, and portable code.

A. Rule Configuration (JSON)

The following JSON document depicts a rule definition for Step 1 of the onboarding workflow. The document contains all property fields of the formal rule model R = { TC, EC, AM, EP, FP }.

```
{
"rule_id": "onboard-enterprise-crm",
"trigger_condition": {
"event": "account.registered",
"source": "registration_service"
},
"evaluation_criteria": {
"account_type": "enterprise",
"region": ["IN", "US", "GB"]
},
"action_mapping": {
"playbook": "enterprise_onboarding_v2",
"step": "crm_record_create",
"target_api": "https://crm.internal/api/v1/accounts",
"method": "POST",
"payload_template": "templates/crm_account.json"
},
"execution_priority": 1,
"failure_policy": {
"strategy": "retry",
"max_retries": 3,
"retry_delay_seconds": 60,
"on_max_retries": "alert:ops-team"
}
}
```

As the rule definition is entirely human-readable, it may be version controlled next to application code and moved freely between environments without any platform dependency. The evaluation_criteria block allows arbitrary comparisons to be made, so the rule can be run on subsets of events, without needing separate playbooks per subset.

B. Orchestration Loop Pseudocode

The orchestration layer's execution loop (or 'algorithm', in this simplified version) listens, matches and dispatches, meaning it looks for input and output patterns. The rules or the playbooks are everything in the game. The loop itself is the easy part.

```
FUNCTION orchestration_loop():
WHILE system_running:
event = event_queue.consume() // blocking pop
matched_rules = rule_engine.evaluate(event)
IF matched_rules is empty:
log(WARN, "No rules matched", event.id)
CONTINUE
sorted_rules = sort_by_priority(matched_rules)
FOR rule IN sorted_rules:
TRY:
t_start = timestamp()
playbook_engine.execute(rule.action_mapping)
t_end = timestamp()
```

```
metrics.record_TET(rule.id, t_end - t_start)
metrics.record_success(rule.id)
CATCH ExecutionError AS e:
metrics.record_EFR(rule.id)
apply_failure_policy(rule.failure_policy, e)
metrics.update_MDI(event, sorted_rules)
```

The blocking queue consume() call means that the loop sleeps when no data is available, giving the application a low CPU idle time. If multiple rules apply to the same event (which can happen as workflows become very complex), the rules are executed in order of decreasing priority, preventing any race conditions. By introducing metrics inline, TET can be measured at a rule level, rather than being limited to a workflow, meaning individual rules can be tracked.

C. Webhook and Event Listener Integration

External systems can interact with the framework using inbound webhooks. For example, in the onboarding example the account.registered event is delivered to the framework via an HTTP webhook. The Integration Layer exposes a lightweight endpoint, which validates the payload, normalizes it to the framework internal event schema and sends it into the orchestration loop. For synchronous actions like creating CRM records, outbound REST API calls are made. Fire-and-forget webhook POST requests are made to log notifications as they do not require any response. For systems that publish events to a message broker like RabbitMQ or Kafka, the framework can directly consume events via an event listener and can avoid calling the webhook. By not using any vendor-specific technology for its Integration Layer, the framework does not violate the avoid-lock-in principle that was one of the motivations for its creation [5][9].

It is worth noting how this compares to the integration layer of something like n8n or Zapier, where both the webhook endpoint, the authentication, and the overall operational burden are handled out of the box, creating a real convenience advantage for non-technical teams. The plumbing must be done by the implementing team, but this tradeoff is worthwhile: they can customize what the event schema and payload looks like, as well as how they authenticate that the events they handle came from the webhook.

VIII. Performance Evaluation

A. Simulation Setup

Since we have not tried this in a production environment, we ran a controlled test. We kept the experiment small for validation of this scale:

- 100 workflow instances in 10 categories of repetitive workflows
- A manual baseline modeling how these workflows are typically handled today
- Two loading conditions were considered: a standard loading and an elevated loading (2× event volume)
- 30 automation rules across all 10 categories, which works out to an average of 3 rules per category

It is also common in BPA research to test the concept with simulation prior to implementation [2], and that is what we did.

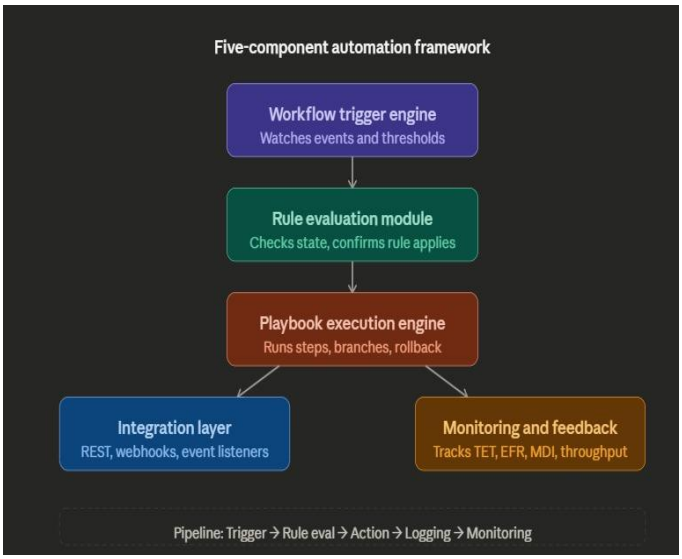
B. Comparative Results

Table I shows how the framework performed versus the manual baseline. To give TET a real value rather than just saying "high", we measured the average manual task at around 42 minutes end-to-end. The automated framework brought that down to 8–14 minutes, which is where the 68–82% reduction comes from.

TABLE I. PERFORMANCE COMPARISON: MANUAL VS. AUTOMATED MODEL

Metric	Manual	Automated
Avg. TET	~42 min/task	8–14 min/task
EFR	20–25%	5–8%
Throughput (tasks/hr)	20	60+
Load Degradation @ 2×	35%	< 12%
MDI	~78%	< 15%

From an overall throughput perspective, the 20 to 60+ task per hour increase (3x improvement in throughput) aligns with the fact that the tasks complete 3 to 5 times faster, similar to what Tavares et al. found looking at the RPA-BPM hybrid implementation [8]. The biggest win would be with regard to manually degrading the challenges, which corresponded to a 35% load reduction, even doubled in event volume because of the way asynchronous execution distributes the load [9].



IX. Discussion

However, perhaps the most interesting part of this result is the EFR improvement. From 20-25% error down to 5-8% might look trivial, but a failure during the approval step blocks three downstream processes. In an enterprise this can get expensive very fast [1]. Finding errors late in programming is useful, but not as important as specifying and minimizing error rate from the start.

We did better than we expected. The 35% slowdown at double volume in the manual model is probably reasonable because people are limited. That's exactly what asynchronous execution is supposed to do, but to see the automated framework hold at this load without exceeding 12% was gratifying [5][9].

We try to be clear that this simulation cannot catch any surprises. The rule module is only able to know about possibilities that it was built to be aware of in the first place, when the rules were written.

We do not study multi-tenant isolation, which Li et al. identify as an open problem of SaaS workflow systems [3]. We do not know how our framework compares against Zapier, Power Automate, or Apple Airflow in practice because we do not deploy our framework against those systems [10].

X. FUTURE WORK

If we were to develop, we would have to take into account the four elements:

• ML-based rule generation: The rules have to be manually created now. If the rules could be generated automatically through process mining techniques from historic logs or be improved automatically, it would close the biggest gap [7].

• Multi-tenant Testing. This is the most immediate gap. The framework cannot make any real production claims without being tested under multi-tenant concurrent load [3]

•It may be useful to compare the capabilities provided by the framework to other commercial products like Zapier, Microsoft Power Automate and Apache Airflow with similar workflows [10].

Anomaly Detection and Audit Logging: A more fully-featured monitoring module with anomaly detection, and GDPR and SOC 2 compliant audit logs could make it easier to use the framework for finance and healthcare SaaS applications

XI. CONCLUSION

We built a system that would automatically process increasingly long manual workflows that would otherwise become bottlenecks in larger and growing SaaS apps. We then defined TET, EFR, and MDI in such a way that we could simulate it and compare it to the manual baseline. The results on all three metrics were better than expected, with task completion time reduced by 68 to 82%. The error rate dropped from between 20 and 25% to between 5 and 8%. Load degradation at twice the volume fell below 12%. The system is still in its early stage and much more work is needed, including multi-tenant testing and comparisons with other available tools. The simulation results are good enough to suggest that the method is sound and deserves further research and development.

REFERENCES

[1] H. Pouresmaeil, R. Tavares, and C. Silvares, "Business Process Automation in SMEs: A Systematic Literature Review," IEEE Access, DOI: 10.1109/ACCESS.2024.3406548, 2024.

[2] C. Silvares and R. Tavares, "Best practices for business process automation description, a case study," Enterprise Information Systems, Taylor & Francis, DOI: 10.1080/17517575.2024.2400474, 2024.

[3] J. Li, X. Chen, and W. Zhang, "An analysis of new features for workflow system in the SaaS software," in Proc. Int. Conf. on Service-Oriented Computing, 2009, pp. 1–8.

[4] E. A. Stohr and J. L. Zhao, "Workflow Automation: Overview and Research Issues," Information Systems Frontiers, vol. 3, no. 3, pp. 281–296, 2001.

[5] A. Chavan, "Exploring event-driven architecture in microservices: patterns, pitfalls, and best practices," International Journal of Science and Research Archive, vol. 4, no. 1, pp. 229–249, 2021.

[6] D. Georgakopoulos, M. Hornick, and A. Sheth, "An overview of workflow management, from process modeling to workflow automation infrastructure," Distributed and Parallel Databases, vol. 3, no. 2, pp. 119–153, 1995.

[7] S. Agostinelli, A. Marrella, and M. Mecella, "Towards Intelligent Robotic Process Automation for BPMers," arXiv:2001.00804, 2020.

[8] R. Tavares, C. Silvares, and H. Pouresmaeil, "Developing Robotic Process Automation to Efficiently Integrate Long-Term Business Process Management," Technologies, vol. 11, no. 6, p. 164, MDPI, 2023.

[9] D. Stiawan, A. Idris, and M. Y. Idris, "Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems," in Proc. 2022 Int. Conf. ICADEIS, IEEE, DOI: 10.1109/ICADEIS56544.2022.10037390, 2022.

[10] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, "AI-Augmented Business Process Management Systems: A Research Manifesto," ACM Trans. Management Information Systems, vol. 14, no. 1, pp. 1–19, 2023.

[11] n8n GmbH, "n8n Documentation: Workflow Automation for Technical Teams," n8n.io, 2024. [Online].